



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Lab experiment-2

2. 8-Queens Problem

Aim

To place eight queens on an 8×8 chessboard so that no two queens attack each other.

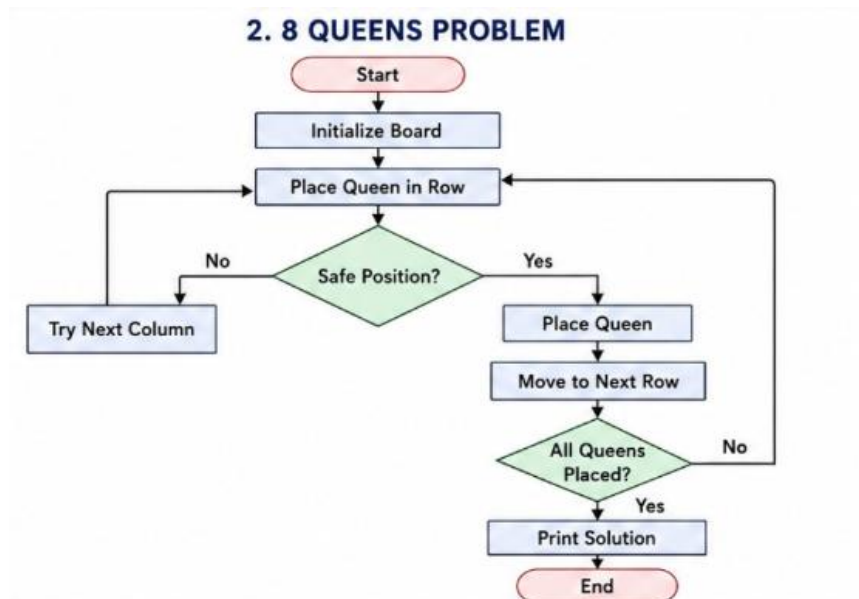
Objective

To find a valid arrangement of eight queens using the Backtracking algorithm.

Algorithm

1. Start from the first row.
2. Place a queen in a safe column.
3. Move to the next row.
4. If no safe position exists, backtrack.
5. Repeat until all queens are placed.
6. Print the solution.

Flowchart



Code

```
from collections import deque
```

```
goal = [1, 2, 3,
```

```
        4, 5, 6,
```

```
        7, 8, 0]
```

```
def get_neighbors(state):
```

```
    neighbors = []
```

```
    index = state.index(0)
```

```
    moves = {
```

```
        0: [1, 3],
```

```
        1: [0, 2, 4],
```

```

2: [1, 5],
3: [0, 4, 6],
4: [1, 3, 5, 7],
5: [2, 4, 8],
6: [3, 7],
7: [4, 6, 8],
8: [5, 7]
}
for move in moves[index]:
    new_state = state[:]
    new_state[index], new_state[move] = new_state[move], new_state[index]
    neighbors.append(new_state)
return neighbors
def solve(start):
    queue = deque([(start, [])])
    visited = []
    while queue:
        state, path = queue.popleft()
        if state == goal:
            return path + [state]
        if state in visited:
            continue
        visited.append(state)
        for neighbor in get_neighbors(state):
            queue.append((neighbor, path + [state]))
    return None

```

```
start = [1, 2, 3,
```

```
        4, 5, 6,
```

```
        0, 7, 8]
```

```
solution = solve(start)
```

```
if solution:
```

```
    print("Solution Steps:")
```

```
    for step in solution:
```

```
        print(step[0:3])
```

```
        print(step[3:6])
```

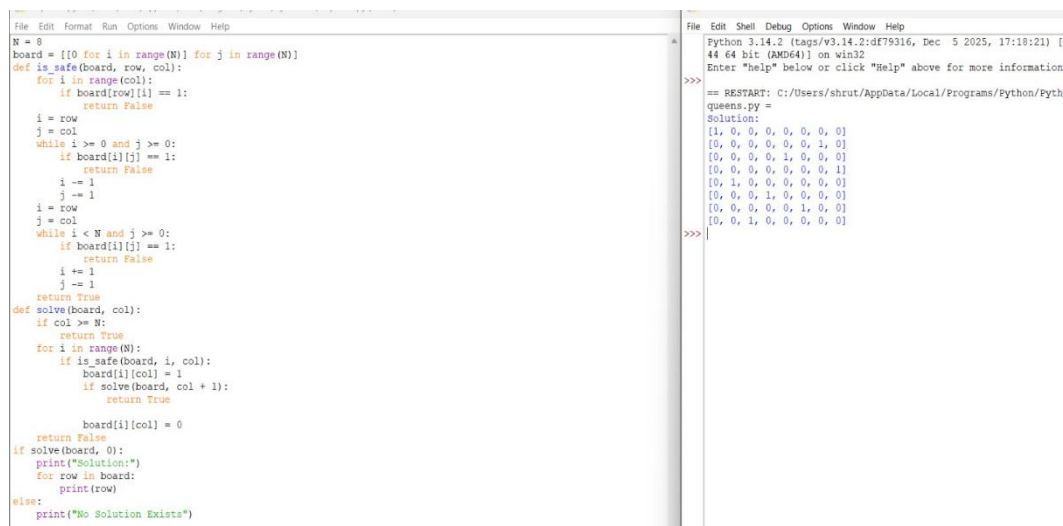
```
        print(step[6:9])
```

```
    print()
```

```
else:
```

```
    print("No Solution Found")
```

Output

The image shows a screenshot of a Python IDE with two windows. The left window displays the source code for an 8-Queens solver using backtracking. The code defines a board, a safe function to check for conflicts, and a solve function that recursively finds a solution. The right window shows the execution output, which includes the restart path, the solution found, and a list of 8 rows representing the queen positions on the board.

```
File Edit Format Run Options Window Help
N = 8
board = [[0 for i in range(N)] for j in range(N)]
def is_safe(board, row, col):
    for i in range(col):
        if board[row][i] == 1:
            return False
    i = row
    j = col
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1
    i = row
    j = col
    while i < N and j >= 0:
        if board[i][j] == 1:
            return False
        i += 1
        j -= 1
    return True
def solve(board, col):
    if col >= N:
        return True
    for i in range(N):
        if is_safe(board, i, col):
            board[i][col] = 1
            if solve(board, col + 1):
                return True
            board[i][col] = 0
    return False
if solve(board, 0):
    print("Solution:")
    for row in board:
        print(row)
else:
    print("No Solution Exists")
```

```
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [
44 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information
>>>
== RESTART: C:/Users/shrut/AppData/Local/Programs/Python/Pyth
queens.py =
Solution:
[1, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 1, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 1]
[0, 1, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 1, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 1, 0, 0]
[0, 0, 1, 0, 0, 0, 0, 0]
>>>
```

Result

The program successfully places all eight queens without conflicts.

Conclusion

Backtracking efficiently finds a valid solution for the 8-Queens problem.